ESPECIFICAÇÃO DO TRABALHO PRÁTICO

Simulador didático de sistemas operacionais

Disciplina de Sistemas Operacionais

Prof. Maurício Acconcia Dias

PROPÓSITO DO PROJETO

Desenvolver, com orientação a objetos, um simulador capaz de representar os principais mecanismos de um sistema operacional. O programa não substituirá o sistema operacional da máquina nem utilizará seus mecanismos internos como objeto principal da simulação. Seu papel será reproduzir, em um ambiente controlado, a evolução de processos e threads, o escalonamento da CPU, a gerência de memória, as operações de entrada e saída e a organização de arquivos.

O trabalho deverá permitir que decisões de projeto sejam observadas e comparadas por meio de eventos, métricas e experimentos reproduzíveis. A qualidade da solução será avaliada tanto pelo funcionamento do simulador quanto pela coerência de sua arquitetura, documentação e análise dos resultados.

Natureza	Desenvolvimento de software e experimentação
Paradigma	Orientação a objetos
Organização	Individual, dupla ou trio, conforme orientação da disciplina
Entregas	Entrega intermediária e entrega final

1. Finalidade e resultados de aprendizagem

O trabalho tem como finalidade consolidar os conteúdos da disciplina mediante a construção de um modelo computacional integrado. Ao concluir o projeto, espera-se que seja possível explicar como as decisões tomadas por um sistema operacional afetam o tempo de resposta, a espera, a utilização dos recursos e o comportamento dos programas simulados.

A implementação deverá evidenciar a relação entre as abstrações estudadas. Um processo manterá recursos e informações de execução; suas threads disputarão a CPU; solicitações de entrada e saída provocarão bloqueios; acessos à memória poderão gerar faltas de página; e operações sobre arquivos deverão respeitar estruturas globais e descritores mantidos por processo. Essas relações devem aparecer no código, no registro de eventos e nos experimentos apresentados no relatório.

Espera-se, especificamente, que o projeto demonstre capacidade de modelar estados e transições, implementar políticas substituíveis, separar responsabilidades entre classes, produzir resultados reproduzíveis e interpretar quantitativamente o comportamento do sistema simulado.

2. Natureza da simulação

O programa será um simulador orientado a eventos discretos. Isso significa que o tempo será representado por um clock lógico, medido em unidades abstratas, e não pelo relógio real do computador. A cada avanço do clock, o núcleo da simulação deverá processar eventos como chegada de uma tarefa, despacho para a CPU, término de quantum, solicitação ou conclusão de entrada e saída, acesso à memória, falta de página e encerramento de uma thread.

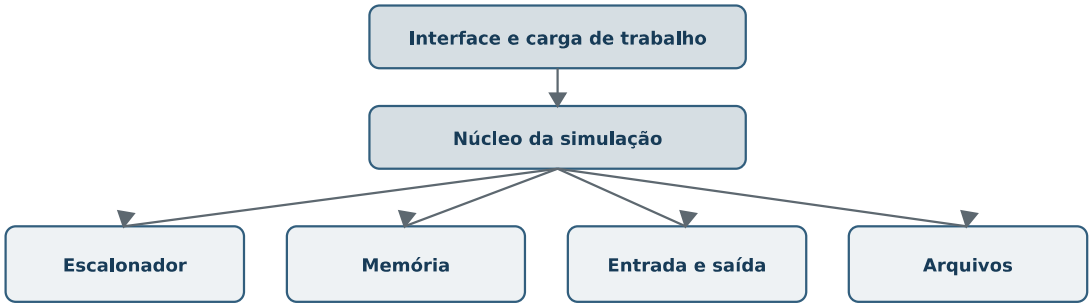
A adoção de um clock lógico evita que o resultado dependa da velocidade do computador utilizado para executar o programa. Também permite repetir exatamente um cenário, comparar políticas sob as mesmas condições e calcular métricas de forma consistente. Caso haja geração aleatória de cargas ou tempos, uma semente deverá ser informada e registrada.

DISTINÇÃO ESSENCIAL

Os processos e as threads do simulador são objetos do modelo. Eles não devem ser confundidos com processos e threads reais criados pela linguagem ou pelo sistema hospedeiro. O paralelismo real não é necessário para o trabalho e não poderá substituir a lógica de escalonamento implementada pelo grupo.

3. Arquitetura esperada

A solução deverá ser organizada em módulos com responsabilidades reconhecíveis. A figura apresenta uma referência conceitual, não uma imposição de nomes de classes. Diferentes arquiteturas serão aceitas desde que mantenham baixo acoplamento, coesão adequada e separação entre mecanismos e políticas.



Todos os módulos registram eventos e métricas no mesmo clock lógico.

O núcleo da simulação deverá coordenar o clock e a fila de eventos. Os demais módulos deverão receber solicitações por interfaces bem definidas. Políticas como algoritmo de escalonamento, substituição de páginas ou seleção de requisições de entrada e saída não devem ficar espalhadas pelo código. Sempre que possível, deverão ser implementadas como componentes substituíveis, permitindo a execução do mesmo cenário com configurações diferentes.

3.1 Linguagem e orientação a objetos

A linguagem poderá ser escolhida pelo grupo, desde que ofereça suporte explícito à orientação a objetos. A implementação deverá empregar encapsulamento, composição, interfaces ou classes abstratas quando contribuírem para a separação de responsabilidades. Herança deverá ser utilizada somente quando houver uma relação conceitual clara entre os tipos envolvidos.

Não será considerada suficiente uma solução que apenas reúna estruturas de dados e funções em uma única classe principal. O modelo deverá tornar identificáveis entidades como processo, thread, PCB, TCB, escalonador, evento, dispositivo, gerenciador de memória, tabela de páginas, arquivo e diretório. Os nomes concretos poderão variar, mas as responsabilidades correspondentes deverão estar representadas.

4. Modelo de execução

O simulador deverá iniciar a partir de uma carga de trabalho descrita em arquivo. Cada tarefa da carga deverá informar, no mínimo, um identificador, o instante de chegada, a prioridade e uma sequência de surtos de CPU e operações de entrada e saída. O formato do arquivo é livre, mas deverá ser textual, validado pelo programa e documentado no README.

4.1 Processos e blocos de controle

Cada processo deverá possuir um identificador único e um PCB. O PCB deverá concentrar as informações necessárias à administração do processo, incluindo estado, prioridade, instante de criação, contador de programa lógico, registradores simulados, referência ao espaço de endereçamento, relação de threads e tabela de arquivos abertos. Não é necessário reproduzir todos os campos de um sistema operacional real, mas cada campo adotado deverá ter função definida no simulador.

O ciclo de vida deverá contemplar os estados novo, pronto, executando, bloqueado e finalizado. Toda mudança de estado deverá ocorrer por um evento identificável e ser registrada no log. Uma transição direta sem causa registrada será considerada falha do modelo.

4.2 Threads e unidades escalonáveis

As threads de usuário serão as unidades que disputam a CPU. Cada thread deverá pertencer a exatamente um processo e possuir um TCB com identificador, estado, referência ao processo, contador de programa lógico, registradores simulados e pilha lógica. Um processo poderá possuir uma ou mais threads. Para cargas simples, um processo com uma única thread deverá funcionar sem tratamento especial.

Quando todas as threads de um processo forem finalizadas, o processo poderá ser encerrado e seus recursos liberados. Uma thread bloqueada por entrada e saída não deverá continuar consumindo CPU. Caso outra thread do mesmo processo esteja pronta, ela poderá ser escalonada normalmente.

4.3 Escalonamento da CPU

O escalonador deverá manter a fila de prontos e selecionar a próxima thread conforme uma política configurável. Serão obrigatórios FCFS, Round Robin e escalonamento por prioridades. O grupo deverá declarar se a política de prioridades é preemptiva ou não preemptiva e implementar corretamente a opção escolhida. No Round Robin, o quantum deverá ser um parâmetro de execução.

A troca de contexto deverá possuir custo configurável. Durante esse intervalo, a CPU estará ocupada com a troca, mas não executará instruções da carga. O simulador deverá contar as trocas realizadas e acumular o tempo gasto com essa sobrecarga. O log deverá distinguir despacho, preempção, bloqueio, retorno à fila de prontos e finalização.

Empates deverão ser resolvidos por uma regra determinística e documentada, como ordem de chegada seguida pelo identificador. A mesma carga, a mesma configuração e a mesma semente deverão produzir o mesmo resultado.

5. Gerenciamento de memória

Cada processo deverá possuir um espaço de endereçamento lógico independente. A implementação mínima será baseada em paginação, com páginas e molduras de tamanho fixo. O tamanho da página, a quantidade de molduras e a sequência de referências à memória deverão ser configuráveis. Cada processo deverá manter uma tabela de páginas capaz de indicar, pelo menos, se a página está presente e em qual moldura foi carregada.

Quando uma referência atingir uma página residente, o acesso será contabilizado como acerto. Quando a página não estiver presente, ocorrerá uma falta de página. A thread responsável deverá ser bloqueada durante o atendimento da falta, e o gerenciador deverá utilizar uma moldura livre ou selecionar uma vítima segundo a política FIFO. Após o tempo de atendimento configurado, a página será carregada, a tabela será atualizada e a thread poderá retornar ao

estado pronto.

A política FIFO constitui o requisito mínimo porque permite que as faltas de página tenham comportamento completo e verificável. LRU ou Clock poderão ser implementadas como extensões. Uma TLB lógica também será opcional. Caso seja incluída, o relatório deverá distinguir acertos na TLB, acertos na tabela de páginas e faltas de página.

CORREÇÃO CONCEITUAL DA ESPECIFICAÇÃO

First fit e best fit não serão utilizados como políticas de alocação de molduras na paginação, pois todas as molduras possuem o mesmo tamanho. Esses algoritmos poderão aparecer somente se o grupo optar, adicionalmente, por modelar alocação contígua ou segmentação com partições de tamanhos variáveis.

6. Entrada e saída

O simulador deverá possuir pelo menos um dispositivo de bloco e um dispositivo de caractere. Cada dispositivo manterá sua própria fila de requisições, seu estado e um tempo de serviço configurável. Quando uma thread solicitar uma operação bloqueante, ela deverá deixar a CPU, passar ao estado bloqueado e permanecer assim até o evento de conclusão. A conclusão deverá gerar uma interrupção simulada e tornar a thread novamente elegível ao escalonamento.

Também deverá existir uma operação não bloqueante. Nesse caso, a solicitação será registrada e a thread poderá prosseguir, recebendo posteriormente o resultado por um mecanismo definido e documentado. Espera ativa é proibida: nenhuma thread poderá permanecer executando apenas para consultar repetidamente se a operação terminou.

A política mínima de atendimento será FCFS por dispositivo. Caso o grupo implemente escalonamento de disco por SSTF ou SCAN, deverá apresentar um experimento comparativo com a mesma sequência de requisições.

7. Sistema de arquivos

O sistema de arquivos deverá representar uma estrutura hierárquica iniciada em um diretório raiz. Diretórios poderão conter arquivos e outros diretórios. Cada objeto persistente deverá possuir um FCB ou nó-i simplificado com nome, tipo, tamanho, permissões básicas, instantes lógicos de criação e modificação e informações necessárias à localização dos blocos de dados.

Deverão ser implementadas as operações de criar arquivo ou diretório, abrir, ler, escrever, fechar, remover e listar conteúdo. A abertura deverá produzir um descritor local ao processo, associado a uma entrada da tabela global de arquivos abertos. O fechamento deverá liberar essa associação. O simulador deverá detectar operações inválidas, como leitura de arquivo fechado, remoção de caminho inexistente ou acesso incompatível com as permissões definidas.

A alocação dos blocos poderá ser contígua ou encadeada. A opção escolhida deverá aparecer no modelo e ser explicada no relatório, incluindo pelo menos uma limitação decorrente da estratégia. Cache de blocos e journaling serão extensões opcionais.

8. Interface, configuração e carga de trabalho

A interface mínima será por linha de comando. Ela deverá permitir selecionar o arquivo de carga, o algoritmo de escalonamento, o quantum, o custo de troca de contexto, o tamanho da página, o número de molduras, os tempos dos dispositivos e a semente de aleatoriedade. Parâmetros inválidos deverão produzir mensagem objetiva e encerrar a execução sem corromper os resultados.

A carga de trabalho deverá ser externa ao código-fonte. Não será aceito alterar classes ou recompilar o programa para mudar processos, chegadas, surtos ou referências de memória. O repositório deverá incluir pelo menos três cargas prontas para execução: uma predominantemente limitada por CPU, uma predominantemente limitada por entrada e saída e uma carga mista.

Cada execução deverá registrar a configuração utilizada no início do log. Uma interface gráfica poderá ser adicionada, mas não substituirá a entrada por arquivo nem os registros textuais necessários à avaliação.

9. Registro de eventos, métricas e experimentos

O log deverá permitir reconstruir o que ocorreu durante a simulação. Cada linha deverá conter o valor do clock, o tipo do evento, os objetos envolvidos e uma descrição concisa. Entre os eventos relevantes estão chegada, admissão, despacho, início e fim de troca de contexto, preempção, bloqueio, interrupção, falta de página, carregamento ou substituição de página, abertura e fechamento de arquivo e finalização.

Ao final, o simulador deverá apresentar as métricas indicadas na tabela. Os tempos deverão ser calculados a partir do clock lógico, com definições aplicadas de maneira uniforme a todas as políticas.

Métrica	Definição operacional
Tempo de retorno	Tempo entre a chegada e a finalização do processo ou da thread analisada.
Tempo de espera	Soma dos intervalos em que a unidade permaneceu na fila de prontos.
Tempo de resposta	Tempo entre a chegada e a primeira utilização da CPU.
Utilização da CPU	Proporção do tempo total em que a CPU executou carga útil; a sobrecarga deve ser informada separadamente.
Utilização dos dispositivos	Proporção do tempo em que cada dispositivo permaneceu atendendo requisições.
Throughput	Quantidade de processos concluídos por unidade de tempo simulado.
Faltas de página	Quantidade e taxa de referências que exigiram carregamento de página.
Trocas de contexto	Quantidade de trocas e tempo total consumido pela sobrecarga configurada.

O relatório final deverá apresentar pelo menos três experimentos. Um deles deverá comparar FCFS, Round Robin e prioridades sobre a mesma carga. Outro deverá analisar o efeito de ao menos dois valores de quantum. O terceiro deverá investigar memória ou entrada e saída. Cada comparação deverá manter constantes os parâmetros que não estiverem sendo estudados e deverá interpretar os resultados, em vez de apenas reproduzir tabelas.

10. Testes e critérios de aceitação

O projeto deverá possuir testes automatizados para regras centrais e testes de integração para cenários completos. Os testes unitários deverão abranger, no mínimo, transições de estado, ordenação das filas de escalonamento, término de quantum, cálculo de métricas, tradução de endereços, ocorrência de falta de página, bloqueio e desbloqueio por entrada e saída e validação das operações básicas de arquivos.

Além dos testes automatizados, o repositório deverá conter pequenas cargas de verificação cujo resultado possa ser conferido manualmente. Uma carga com duas ou três threads e poucos eventos é mais adequada para essa finalidade do que uma carga extensa. O README deverá informar o comando de execução dos testes e o resultado esperado.

Componente	Condição mínima de aceitação
Execução	O programa inicia por comando documentado, lê a carga externa e termina sem intervenção manual.
Determinismo	Execuções com os mesmos parâmetros e semente geram a mesma sequência de eventos e métricas.
Escalonamento	As três políticas obrigatórias podem ser selecionadas sem alteração do código.
Memória	A tradução, a falta de página e a substituição FIFO aparecem no log e alteram corretamente o estado da thread.
Entrada e saída	Operações bloqueantes removem a thread da CPU; a conclusão gera evento e retorno ao estado pronto.
Arquivos	As operações mínimas respeitam caminhos, descritores, abertura e fechamento.
Resultados	As métricas são apresentadas por execução e sustentam os experimentos do relatório.

11. Entregáveis

O projeto deverá ser mantido em um repositório GitHub acessível ao professor. O histórico de commits deverá refletir a evolução do trabalho e a participação dos integrantes. Arquivos gerados, dependências locais, credenciais e dados temporários não deverão ser versionados.

Etapa	Conteúdo obrigatório
Entrega intermediária	Repositório estruturado; descrição da arquitetura; diagrama de classes preliminar; núcleo do clock e dos eventos; processos, threads e ao menos uma política de escalonamento funcionando; testes iniciais; relatório de progresso com funcionalidades concluídas, percentual estimado, limitações conhecidas e plano para a etapa final.
Entrega final	Código completo; três políticas de escalonamento; memória paginada; entrada e saída; sistema de arquivos; cargas de trabalho; testes; log; métricas; documentação; diagrama de classes atualizado; relatório técnico com experimentos e análise dos resultados.

11.1 Conteúdo do repositório

- Código-fonte organizado em diretórios e sem arquivos binários desnecessários.
- README com requisitos, instalação, compilação, execução, parâmetros, formato da carga e exemplos de comandos.
- Diagrama de classes coerente com a versão entregue do código.
- Cargas de trabalho de exemplo e arquivos de configuração necessários à reprodução dos experimentos.

- Testes automatizados e instruções para executá-los.
- Relatório técnico final em PDF, com decisões de projeto, limitações, experimentos, tabelas e interpretação das métricas.

11.2 Relatório técnico

O relatório deverá explicar a arquitetura implementada e justificar as principais decisões. Deverá apresentar a modelagem dos estados, o funcionamento das políticas, a forma de cálculo das métricas, os cenários experimentais e a análise dos resultados. Trechos extensos de código não deverão ser usados para substituir explicações. Diagramas, tabelas e excertos curtos poderão ser incluídos quando contribuírem para a compreensão.

12. Critérios de avaliação

Critério	Peso	Aspectos observados
Correção funcional e aderência	30%	Atendimento ao escopo, consistência dos estados, eventos, políticas e operações obrigatórias.
Projeto orientado a objetos	25%	Coesão, acoplamento, encapsulamento, separação entre mecanismo e política e clareza arquitetural.
Qualidade do código	20%	Legibilidade, organização, tratamento de erros, testes, documentação e histórico do repositório.
Experimentos e métricas	15%	Reprodutibilidade, controle de variáveis, correção das métricas e interpretação dos resultados.
Relatório técnico	10%	Precisão conceitual, justificativas, descrição da implementação e qualidade da apresentação.

Funcionalidades apenas declaradas, mas não demonstradas durante a avaliação, não serão consideradas concluídas. Erros que impeçam a execução do programa ou a reprodução dos experimentos poderão comprometer mais de um critério, de acordo com seu impacto sobre o trabalho.

13. Extensões opcionais

Extensões poderão acrescentar até 10% à nota do trabalho, respeitado o limite máximo definido para a disciplina. O bônus será concedido somente quando o escopo obrigatório estiver funcional e a extensão estiver integrada, testada, documentada e analisada. A simples presença de classes ou opções de menu não caracteriza uma extensão concluída.

- Substituição de páginas por LRU ou Clock, acompanhada de comparação com FIFO.
- Prioridades com envelhecimento para reduzir inanição.
- Escalonamento de disco por SSTF ou SCAN, com análise do deslocamento total.
- Cache de blocos ou journaling simplificado no sistema de arquivos.
- Interface gráfica com linha do tempo, estados das threads, filas e ocupação da memória.

14. Restrições e limites do projeto

Não é necessário implementar multiprocessamento real, proteção em modo usuário e modo núcleo, chamadas de sistema reais, drivers do sistema hospedeiro, persistência física dos arquivos simulados ou comunicação direta com o hardware. Esses recursos estão fora do escopo porque o trabalho se concentra na modelagem dos mecanismos e na observação de suas consequências.

Bibliotecas poderão ser utilizadas para leitura de arquivos, construção da interface, geração de gráficos, testes e estruturas de apoio. Não será permitido utilizar uma biblioteca que já implemente o núcleo do simulador, o escalonamento, a paginação ou o sistema de arquivos exigido. Dependências externas deverão ser declaradas e justificadas no README.

15. Regras acadêmicas

O código e o relatório deverão ser produzidos pelo grupo. Todo material externo utilizado deverá ser identificado. Bibliotecas, exemplos, imagens, textos, algoritmos adaptados e ferramentas de apoio deverão ser informados conforme as orientações da disciplina. O grupo deverá compreender e ser capaz de explicar integralmente a solução entregue.

Plágio, reutilização não autorizada de trabalhos, apresentação de código cuja autoria não possa ser explicada ou omissão deliberada de fontes implicará aplicação das normas institucionais, incluindo a possibilidade de nota zero. A avaliação poderá incluir demonstração, perguntas individuais e solicitação de pequenas modificações durante a apresentação para verificar o domínio do projeto.

16. Lista de conferência para a entrega final

- O repositório pode ser obtido e executado a partir das instruções do README.
- A carga de trabalho é externa ao código e possui validação de formato.
- O clock lógico e a fila de eventos controlam toda a simulação.
- Processos, threads, PCB, TCB e transições de estado estão representados.
- FCFS, Round Robin e prioridades podem ser selecionados por configuração.
- A paginação trata faltas de página e substituição FIFO.
- Os dispositivos possuem filas e produzem interrupções simuladas.
- O sistema de arquivos implementa as operações e tabelas obrigatórias.
- O log permite acompanhar eventos relevantes em ordem temporal.
- As métricas são calculadas e utilizadas em pelo menos três experimentos.
- Os testes automatizados e as cargas de verificação estão incluídos.
- O diagrama, o código e o relatório descrevem a mesma arquitetura.

CONDIÇÃO DE ENTREGA

Antes do envio, o grupo deverá realizar uma execução completa em ambiente limpo, utilizando somente as instruções registradas no repositório. A impossibilidade de iniciar o programa, localizar uma dependência ou reproduzir uma carga será considerada problema da entrega.